

ORDER MANAGEMENT SYSTEM

1. Introduction:

The Order Management System is a database project developed using Oracle SQL. It stores and manages information about customers, orders, products, and order details.

2. Creating ORDERS Table:

```
SQL> CREATE TABLE ORDERS(ORDER_ID VARCHAR2(7), CUSTOMER_ID NUMBER, ORDER_DATE DATE, TOTAL_AMOUNT NUMBER(10,2), ORDER_STATUS VARCHAR2(20));
```

Table created.

```
SQL> DESC ORDERS;
```

Name	Null?	Type
ORDER_ID		VARCHAR2(7)
CUSTOMER_ID		NUMBER
ORDER_DATE		DATE
TOTAL_AMOUNT		NUMBER(10,2)
ORDER_STATUS		VARCHAR2(20)

This query creates the **ORDERS** table to store order ID, customer ID, order date, total amount, and order status.

3. Creating ORDER_DETAILS Table:

```
SQL> CREATE TABLE ORDER_DETAILS(ORDER_DETAIL_ID VARCHAR2(10),ORDER_ID VARCHAR2(7),PRODUCT_ID VARCHAR2(10),QUANTITY NUMBER,PRICE NUMBER(10,2));
```

Table created.

```
SQL> DESC ORDER_DETAILS;
```

Name	Null?	Type
ORDER_DETAIL_ID		VARCHAR2(10)
ORDER_ID		VARCHAR2(7)
PRODUCT_ID		VARCHAR2(10)
QUANTITY		NUMBER
PRICE		NUMBER(10,2)

This table stores the products included in each order, along with quantity and price.

4. Adding Primary Keys:

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT PK_ORDERS PRIMARY KEY(ORDER_ID);
```

```
Table altered.
```

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT PK_ORDER_DETAIL_ID PRIMARY KEY(ORDER_DETAIL_ID);
```

```
Table altered.
```

Primary keys uniquely identify each record in the respective tables.

5. Adding Foreign Keys:

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT FK_ORDERS_CUSTOMER FOREIGN KEY(CUSTOMER_ID) REFERENCES CUSTOMER(CUSTOMER_ID);
```

```
Table altered.
```

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_ORDER_DETAILS_ORDER FOREIGN KEY(ORDER_DETAIL_ID) REFERENCES ORDERS(ORDER_ID);
```

```
Table altered.
```

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_ORDER_DETAILS_PRODUCT FOREIGN KEY(PRODUCT_ID) REFERENCES PRODUCT(PRODUCT_ID);
```

```
Table altered.
```

Foreign keys establish relationships between customers, orders, order details, and products.

6. Adding NOT NULL Constraints:

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_ORDER_DETAILS_PRODUCT FOREIGN KEY(PRODUCT_ID) REFERENCES PRODUCT(PRODUCT_ID);
```

```
Table altered.
```

```
SQL> ALTER TABLE ORDERS MODIFY CUSTOMER_ID CONSTRAINT NN_ORDERS_CUSTOMER NOT NULL;
```

```
Table altered.
```

```
SQL> ALTER TABLE ORDERS MODIFY ORDER_DATE CONSTRAINT NN_ORDERS_DATE NOT NULL;
```

```
Table altered.
```

These constraints ensure that customer ID and order date cannot be left empty.

7. Adding CHECK Constraints:

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_ORDER_QUANTITY CHECK(QUANTITY>0);
```

```
Table altered.
```

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_ORDER_PRICE CHECK(PRICE >0);
```

```
Table altered.
```

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_ORDER_AMOUNT CHECK(TOTAL_AMOUNT >0);
```

```
Table altered.
```

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_ORDER_STATUS CHECK(ORDER_STATUS IN ('PENDING','SHIPPED','DELEIVERED'));
```

```
Table altered.
```

These constraints ensure that quantity, price, and total amount are positive and that only valid order statuses are accepted.

8. Inserting Orders:

```
SQL> INSERT INTO ORDERS
  2  VALUES ('&ORDER_ID', &CUSTOMER_ID,
  3          TO_DATE('&ORDER_DATE', 'DD-MM-YYYY'),
  4          &TOTAL_AMOUNT, '&ORDER_STATUS');
```

```
Enter value for order_id: 001
```

```
Enter value for customer_id: 1
```

```
Enter value for order_date: 28-09-2026
```

```
Enter value for total_amount: 2800
```

```
Enter value for order_status: DELIVERED
```

```
1 row created.
```

This query inserts order details into the **ORDERS** table. The **&** symbol accepts values from the user.

9. Displaying Orders:

```
SQL> SELECT * FROM ORDERS;
```

ORDER_I	CUSTOMER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
0001	1	01-SEP-26	2500	DELIVERED
0002	2	02-SEP-26	1800	SHIPPED
0003	1	03-SEP-26	3200	DELIVERED
0004	3	04-SEP-26	1500	PENDING
0005	4	05-SEP-26	4200	DELIVERED
0006	2	06-SEP-26	2750	SHIPPED
0007	5	07-SEP-26	3600	DELIVERED
0008	6	08-SEP-26	1200	PENDING
0009	1	09-SEP-26	5100	DELIVERED
0010	7	10-SEP-26	2300	SHIPPED
0011	8	11-SEP-26	1900	DELIVERED
0012	3	12-SEP-26	3400	PENDING
0013	9	13-SEP-26	2800	SHIPPED
0014	10	14-SEP-26	4500	DELIVERED
0015	4	15-SEP-26	1600	PENDING

15 rows selected.

Displays all records from the **ORDERS** table.

10. Inserting Order Details:

```
SQL> INSERT INTO ORDER_DETAILS VALUES('&ORDER_DETAIL_ID','&ORDER_ID','&PRODUCT_ID',&QUANTITY,&PRICE);
Enter value for order_detail_id: OD001
Enter value for order_id: 0001
Enter value for product_id: P001
Enter value for quantity: 1
Enter value for price: 2500
old 1: INSERT INTO ORDER_DETAILS VALUES('&ORDER_DETAIL_ID','&ORDER_ID','&PRODUCT_ID',&QUANTITY,&PRICE)
new 1: INSERT INTO ORDER_DETAILS VALUES('OD001','0001','P001',1,2500)

1 row created.
```

This query inserts product and quantity details for an order.

11. Displaying Order Details:

```
SQL> SELECT * FROM ORDER_DETAILS;
```

ORDER_D	ORDER_I	PRODUCT	QUANTITY	PRICE
OD001	0001	P001	1	2500
OD002	0002	P002	2	900
OD003	0003	P003	1	3200
OD004	0004	P004	3	500
OD005	0005	P005	2	2100
OD006	0006	P006	1	2750
OD007	0007	P007	2	1800
OD008	0008	P008	1	1200
OD009	0009	P009	3	1700
OD010	0010	P010	1	2300
OD011	0011	P011	2	950
OD012	0012	P012	1	3400
OD013	0013	P013	1	4500
OD014	0014	P014	1	4500
OD015	0015	P015	2	800

```
15 rows selected.
```

Displays all records from the **ORDER_DETAILS** table.

12. Customer and Order Details:

```
SQL> SELECT c.CUSTOMER_NAME,  
2         o.ORDER_ID,  
3         o.ORDER_DATE,  
4         o.TOTAL_AMOUNT,  
5         o.ORDER_STATUS  
6 FROM CUSTOMER c  
7 JOIN ORDERS o  
8 ON c.CUSTOMER_ID = o.CUSTOMER_ID  
9 ORDER BY c.CUSTOMER_NAME, o.ORDER_DATE;
```

CUSTOMER_NAME	ORDER_I	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
Ajay Kumar	0011	11-SEP-26	1900	DELIVERED
Anjali Devi	0007	07-SEP-26	3600	DELIVERED
Arun Kumar	0001	01-SEP-26	2500	DELIVERED
Arun Kumar	0003	03-SEP-26	3200	DELIVERED
Arun Kumar	0009	09-SEP-26	5100	DELIVERED
Karthik M	0005	05-SEP-26	4200	DELIVERED
Karthik M	0015	15-SEP-26	1600	PENDING
Manoj R	0013	13-SEP-26	2800	SHIPPED
Meeena K	0010	10-SEP-26	2300	SHIPPED
Nithya S	0014	14-SEP-26	4500	DELIVERED
Priya Sharma	0002	02-SEP-26	1800	SHIPPED
Priya Sharma	0006	06-SEP-26	2750	SHIPPED
Rahul Raj	0004	04-SEP-26	1500	PENDING
Rahul Raj	0012	12-SEP-26	3400	PENDING
Vijay Kumar	0008	08-SEP-26	1200	PENDING

15 rows selected.

This query joins the **CUSTOMER** and **ORDERS** tables to display customer names along with their order information.

13. Product-wise Order Analysis:

```
SQL> SELECT p.PRODUCT_NAME,  
2         COUNT(od.ORDER_ID) AS ORDER_COUNT,  
3         SUM(od.QUANTITY) AS TOTAL_QUANTITY  
4 FROM PRODUCT p  
5 JOIN ORDER_DETAILS od  
6 ON p.PRODUCT_ID = od.PRODUCT_ID  
7 GROUP BY p.PRODUCT_NAME  
8 ORDER BY TOTAL_QUANTITY DESC;
```

PRODUCT_NAME	ORDER_COUNT	TOTAL_QUANTITY
Running Shoes	1	3
Cotton Shirt	1	3
Yoga Mat	1	2
Building Blocks	1	2
Python programming	1	2
Denim Jeans	1	2
Wireless Mouse	1	2
Study Table	1	1
Electric Kettle	1	1
Bluetooth Speaker	1	1
Data Structures	1	1
Office Chair	1	1
Cooking oil	1	1
Face wash	1	1
Smart phone	1	1

15 rows selected.

This query shows each product, the number of orders containing it, and the total quantity ordered.

14. Customer Spending Analysis:

```
SQL> SELECT c.CUSTOMER_NAME,  
2      COUNT(o.ORDER_ID) AS TOTAL_ORDERS,  
3      SUM(o.TOTAL_AMOUNT) AS TOTAL_SPENDING,  
4      ROUND(AVG(o.TOTAL_AMOUNT), 2) AS AVERAGE_ORDER_VALUE  
5 FROM CUSTOMER c  
6 JOIN ORDERS o  
7 ON c.CUSTOMER_ID = o.CUSTOMER_ID  
8 GROUP BY c.CUSTOMER_NAME  
9 ORDER BY TOTAL_SPENDING DESC;
```

CUSTOMER_NAME	TOTAL_ORDERS	TOTAL_SPENDING	AVERAGE_ORDER_VALUE
Arun Kumar	3	10800	3600
Karthik M	2	5800	2900
Rahul Raj	2	4900	2450
Priya Sharma	2	4550	2275
Nithya S	1	4500	4500
Anjali Devi	1	3600	3600
Manoj R	1	2800	2800
Meeena K	1	2300	2300
Ajay Kumar	1	1900	1900
Vijay Kumar	1	1200	1200

10 rows selected.

This query calculates the number of orders, total spending, and average order value for each customer.

15. Conclusion:

The **Order Management System** was successfully implemented using Oracle SQL. The project demonstrates table creation, constraints, data insertion, joins, grouping, sorting, and aggregate functions.

The database can be used to efficiently manage **customers, products, orders, and order details**.